FIAP — Faculdade de Informática e Administração Paulista
Domain Driven Design using Java — Entrega da 4ª Sprint

SoulMove

API REST em Spring Boot — mobilidade urbana sustentável com recompensas em Pontos Soul Up

RM	Integrante
571046	Allyson Victor
569103	Carlos Eduardo Oliveira Silva
570294	Kaique Ziantoni
573058	Marcus Vinicius Costa
571506	Silas Oliveira

Equipe: **Un-glitch Tech**
Solução: **SoulMove**
Turma: **1TDSPV**

São Paulo — 2026

Sumário

1. Objetivo e escopo do projeto
2. Funcionalidades implementadas
3. Tabela de endpoints da API REST
4. Protótipo — prints das telas
5. Modelo de Entidade-Relacionamento (MER)
6. Diagrama de classes atualizado
7. Procedimentos para rodar a aplicação
8. Deploy na nuvem e repositório

1. Objetivo e escopo do projeto

O SoulMove é a solução da equipe Un-glitch Tech para o Challenge Soul Up. Ele incentiva a mobilidade urbana sustentável recompensando quem usa transporte público: cada viagem registrada passa por uma validação antifraude, tem o CO_2 evitado calculado em relação ao mesmo trajeto feito de carro e o resultado é convertido em Pontos Soul Up, que ficam na carteira do usuário e podem ser resgatados.

Na 3ª Sprint o domínio foi implementado em Java com as camadas Model, Connection e DAO, exercitadas por uma classe de teste. Nesta 4ª Sprint a aplicação foi transformada em uma **API RESTful com Spring Boot**, acrescentando a camada BO (regras de negócio e validações), a camada Resource (endpoints HTTP), o filtro de CORS, o tratamento centralizado de exceções e a integração com o site React do projeto, que passou a ler e gravar dados pelo banco através da API.

Dentro do escopo

- CRUD completo de usuários, meios de transporte (ônibus e metrô) e viagens, persistido no Oracle da FIAP.
- Regras de negócio da Sprint 3 aplicadas pela API: validação de CPF, SoulMove Engine (score de confiabilidade da viagem), cálculo de CO_2 evitado por modal e crédito/estorno/resgate de pontos.
- Endpoints REST com verbos e códigos de status adequados, respostas de erro em JSON e CORS liberado para o front-end.
- Página *Painel* no site React consumindo a API (listagem de usuários, transportes e viagens; cadastro de usuário; registro e exclusão de viagem).
- Deploy em container Docker no Render, com as credenciais do banco em variáveis de ambiente.

Fora do escopo

- Autenticação e perfis de acesso (login).
- Integração com a API Olho Vivo da SPTrans e com sistemas de bilhetagem.
- Catálogo de recompensas e resgate por cupom (previstos na modelagem de banco, não expostos nesta API).

2. Funcionalidades implementadas

Funcionalidade	Onde está no código	Regra aplicada
Cadastro, consulta, alteração e exclusão de usuários	`UsuarioResource` → `UsuarioBO` → `UsuarioDAO`	Nome com 3+ caracteres, CPF válido (dígitos verificadores) e único, e-mail válido, bilhete obrigatório. Cadastro novo nasce ativo e com saldo zero. Usuário com viagens não pode ser excluído.
Cadastro, consulta, alteração e exclusão de meios de transporte	`MeioTransporteResource` → `MeioTransporteBO` → `MeioTransporteDAO`	`tipoModal` ONIBUS ou METRO (o BO instancia a subclasse correta); consumo maior que zero; fator de emissão não negativo. Transporte com viagens não pode ser excluído.
Registro de viagem com cálculo de recompensa	`ViagemResource` → `ViagemBO` → `Viagem.processarRecompensa()`	Distância e duração positivas, data não futura, usuário ativo. O SoulMove Engine calcula o score pela velocidade média do modal; viagem validada gera CO_2 evitado × 100 pontos, creditados no usuário.
Alteração e exclusão de viagem	`ViagemBO.atualizar()` / `remover()`	Ao alterar, a recompensa é recalculada e o saldo ajustado; ao excluir, os pontos da viagem são estornados.
Resgate de Pontos Soul Up	`POST /usuarios/{id}/resgates` → `Usuario.resgatarPontos()`	Só para cadastro ativo e com saldo suficiente.
Calculadora de impacto do site	`ImpactoResource`	Mesma fórmula da página inicial: km/semana × 48 semanas × 0,09 kg CO_2, árvores equivalentes e pontos.
Tratamento de erros	`TratadorDeExcecoes` (`@RestControllerAdvice`)	Validação → 400; não encontrado → 404; método inválido → 405; erro de banco → 500. Resposta sempre em JSON: `{"status": "...", "erro": "..."}`.
CORS	`CorsFilter` (`@Component implements Filter`)	Libera GET, POST, PUT, DELETE e OPTIONS para o front-end hospedado em outro domínio.
Status da API	`StatusResource`	`GET /` informa versão, banco em uso e se a conexão está disponível (usado no health check do deploy).

3. Tabela de endpoints da API REST

Base local: `http://localhost:8080`. Todas as respostas são `application/json`. Os códigos de erro valem para todos os endpoints: **400** dados inválidos, **404** recurso ou rota inexistente, **405** verbo não suportado, **500** falha no banco.

Verbo	URI	Descrição	Corpo da requisição	Sucesso	Erros
GET	/	Status da API e do banco	—	200	—
GET	/usuarios	Lista todos os usuários	—	200	500
GET	/usuarios/{id}	Busca usuário por id	—	200	404, 500
POST	/usuarios	Cadastra usuário	`{nome, cpf, email, numeroBilheteTransporte}`	201	400, 500
PUT	/usuarios/{id}	Altera usuário (saldo não é alterado)	`{nome, cpf, email, numeroBilheteTransporte, cadastroAtivo}`	200	400, 404, 500
DELETE	/usuarios/{id}	Remove usuário sem viagens	—	204	400, 404, 500
POST	/usuarios/{id}/resgates	Resgata pontos do saldo	`{pontos}`	200	400, 404, 500
GET	/transportes	Lista meios de transporte	—	200	500
GET	/transportes/{id}	Busca meio de transporte	—	200	404, 500
POST	/transportes	Cadastra ônibus ou metrô	`{tipoModal, identificacao, combustivelUtilizado, fatorEmissaoCo2, consumo, participaDoPrograma}`	201	400, 500
PUT	/transportes/{id}	Altera meio de transporte	mesmo corpo do POST	200	400, 404, 500
DELETE	/transportes/{id}	Remove meio de transporte sem viagens	—	204	400, 404, 500
GET	/viagens	Lista viagens; `?idUsuario=N` filtra por usuário	—	200	404, 500

GET	/viagens/{id}	Busca viagem (com usuário e transporte)	—	200	404, 500
POST	/viagens	Registra viagem, valida, calcula CO_2 e credita pontos	`{idUsuario, idTransporte, distanciaEmKm, duracaoEmMinutos, dataViagem}`	201	400, 404, 500
PUT	/viagens/{id}	Altera viagem e recalcula a recompensa	mesmo corpo do POST	200	400, 404, 500
DELETE	/viagens/{id}	Remove viagem e estorna os pontos	—	204	404, 500
GET	/impacto?kmSemanais=N	Calculadora de impacto anual	—	200	400

Exemplos

```
POST /viagens
{"idUsuario": 1, "idTransporte": 1, "distanciaEmKm": 12, "duracaoEmMinutos": 22,
"dataViagem": "2026-10-20"}

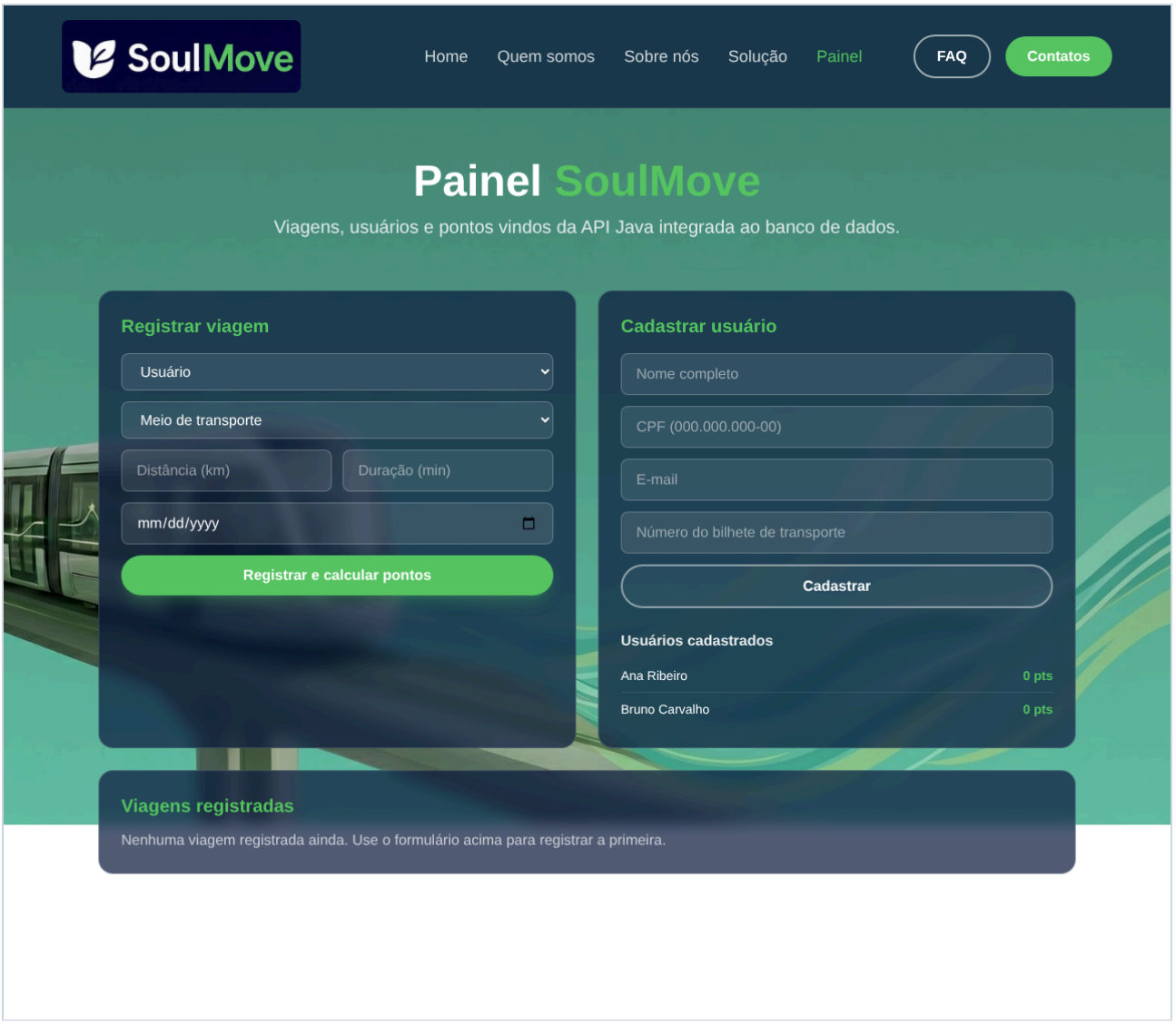
201 Created
{"idViagem": 1, "usuario": {"idUsuario": 1, "nome": "Ana Ribeiro", "saldoPontosSoulUp":
222, ...},
 "meioTransporte": {"idTransporte": 1, "tipoModal": "METRO", "identificacao": "Linha 2 -
Verde", ...},
 "distanciaEmKm": 12.0, "duracaoEmMinutos": 22, "dataViagem": "2026-10-20",
 "co2EvitadoKg": 2.2206, "pontosGerados": 222, "validada": true}

POST /usuarios  com CPF "111.111.111-11"
400 Bad Request
{"status": "400", "erro": "CPF invalido."}

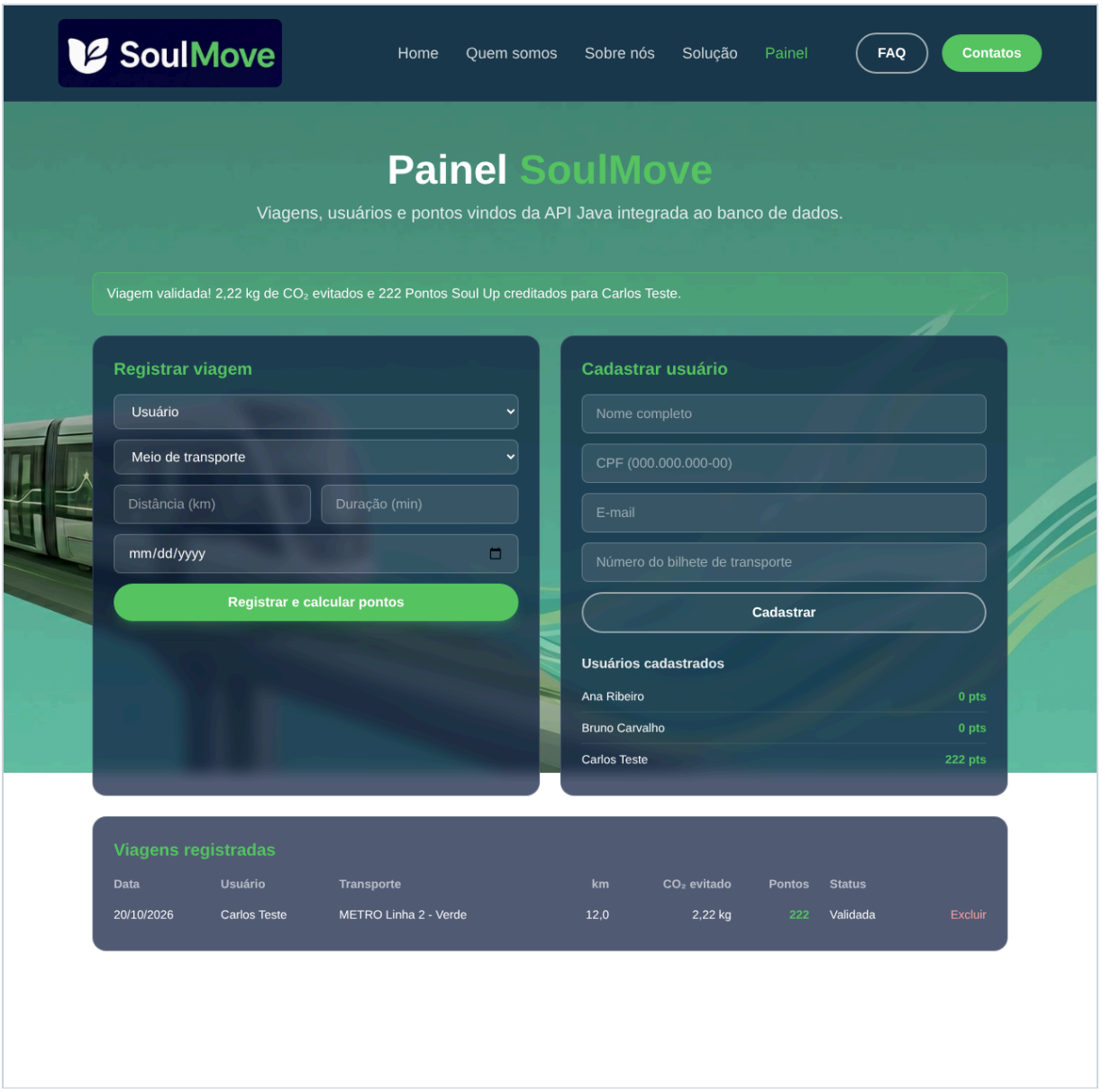
GET /usuarios/9999
404 Not Found
{"status": "404", "erro": "Usuario 9999 nao encontrado."}
```

4. Protótipo — prints das telas

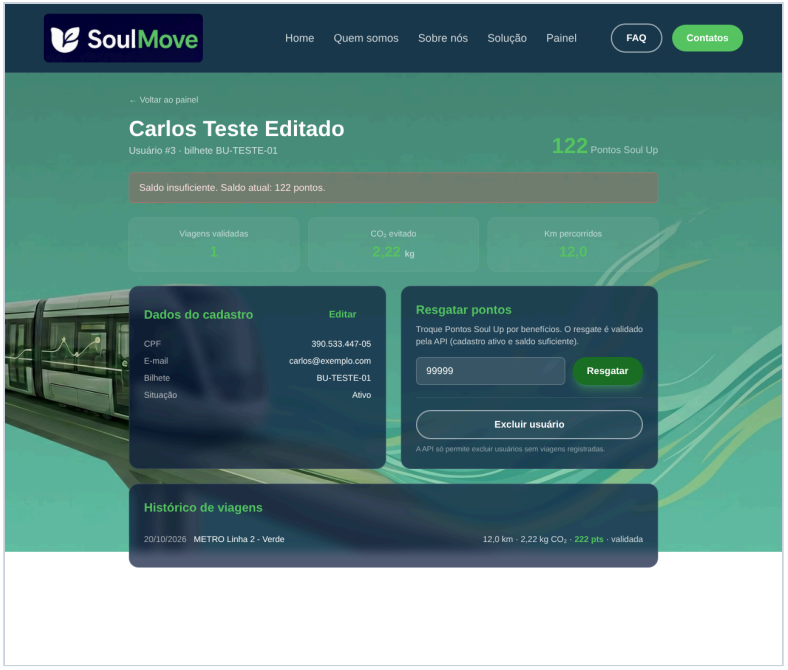
As telas abaixo são do site React do SoulMove (repositório *Sprint-Front-FIAP*) consumindo a API Java. A página **Painel** foi criada nesta Sprint para a integração: ela lê usuários, transportes e viagens do banco e envia cadastros e registros de viagem para a API.



Tela 1 — Painel ao abrir: formulários de registro de viagem e cadastro de usuário, lista de usuários com saldo (GET /usuarios, GET /transportes, GET /viagens).



Tela 2 — Após o POST /viagens: a API validou a viagem de ônibus de 9 km em 35 min, calculou 1,47 kg de CO_2 evitados e creditou 147 pontos; a tabela e o saldo foram atualizados a partir do banco.



Tela 3 — Rota dinâmica /usuarios/{id}: GET /usuarios/3 e GET /viagens?idUsuario=3; o nome foi alterado via PUT, 100 pontos resgatados via POST /usuarios/3/resgates e a tentativa de resgate acima do saldo exibe o erro HTTP 400 devolvido pela API.



Tela 4 — Página inicial do site com o novo item "Painel" no menu. A calculadora de impacto tem a mesma fórmula exposta em GET /impacto.

> **Deploy:** inserir aqui o print do endpoint `GET /` da API publicada no Render, mostrando `"bancoDisponivel": true`.

5. Modelo de Entidade-Relacionamento (MER)

Mesma estrutura da Sprint 3, no Oracle da FIAP: um usuário realiza várias viagens e um meio de transporte é utilizado em várias viagens. Ônibus e metrô ficam em uma única tabela, diferenciados pela coluna `TP_MODAL`. As chaves primárias são geradas por sequences (`SQ_SOULMOVE_*`).

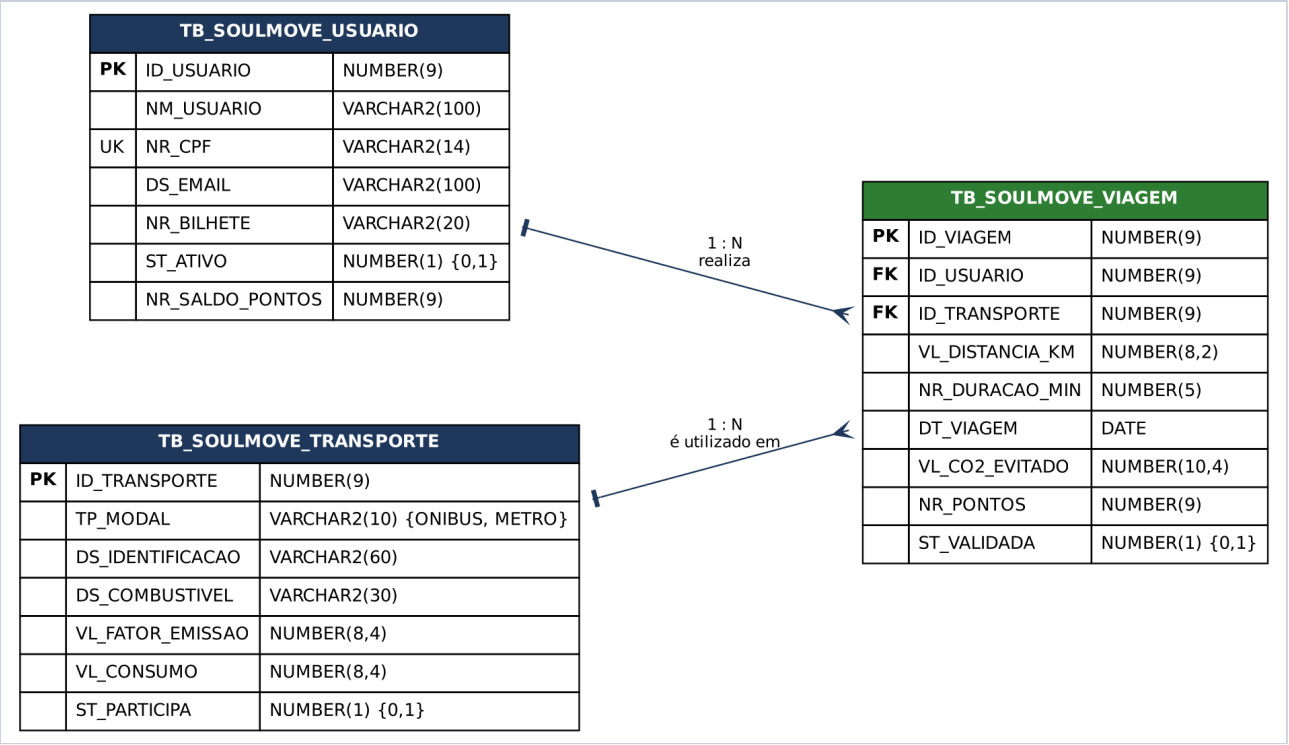


Figura 1 — MER: TB_SOULMOVE_USUARIO (1:N) TB_SOULMOVE_VIAGEM (N:1) TB_SOULMOVE_TRANSPORTE.

6. Diagrama de classes atualizado

O domínio da Sprint 3 foi mantido: `Usuario` herda de `Pessoa`; `Onibus` e `Metro` especializam a classe abstrata `MeioTransporte`, cada um com o seu cálculo de emissão; `Viagem` associa um usuário a um meio de transporte e concentra as regras de validação e recompensa.

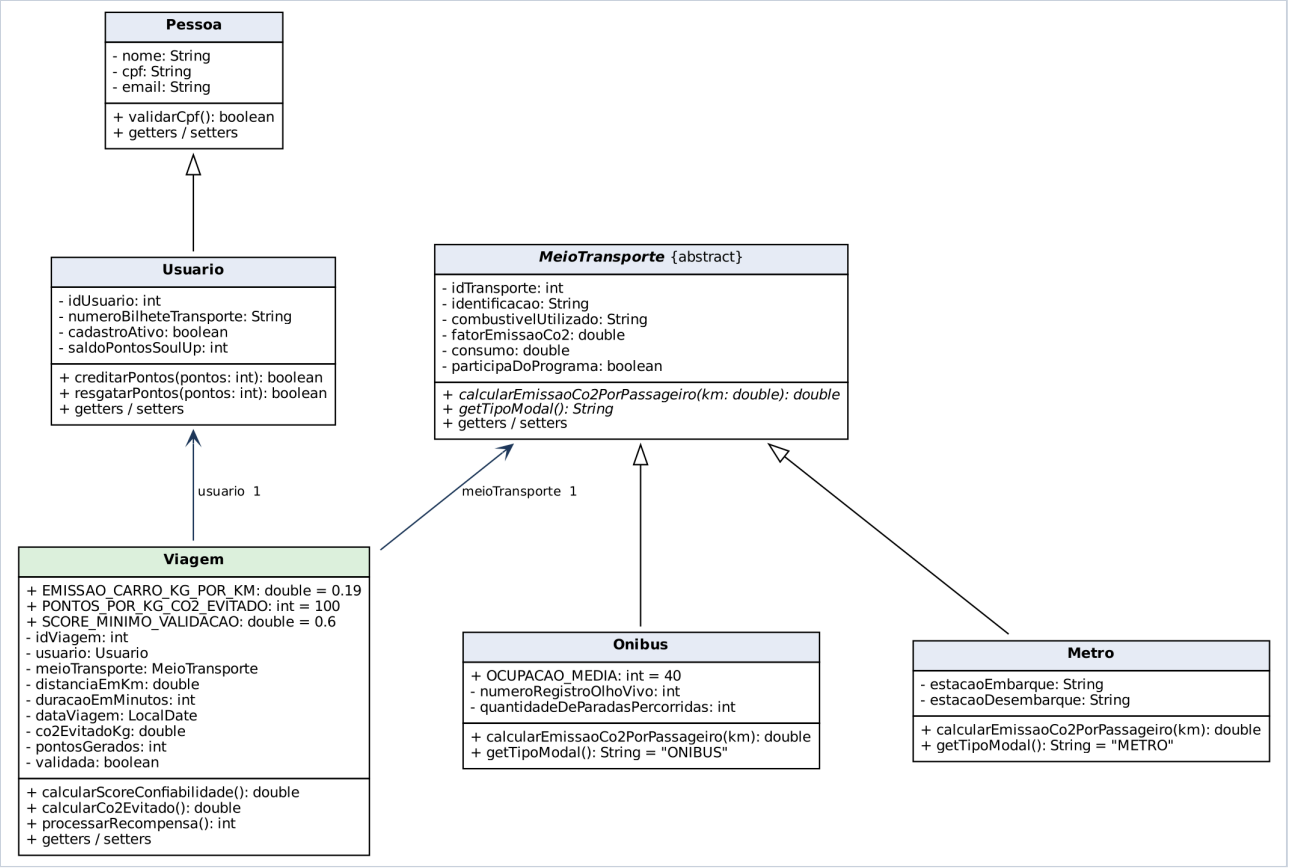


Figura 2 — Classes do domínio (pacote `model`).

As demais camadas da API: cada **Resource** recebe a requisição HTTP e chama o **BO** correspondente; o BO valida os dados, aplica as regras e usa os **DAOs**, que abrem a conexão pela **ConnectionFactory**. Os **DTOs** representam o JSON recebido quando ele não corresponde diretamente a uma classe do modelo (por exemplo, a viagem chega com os ids do usuário e do transporte).

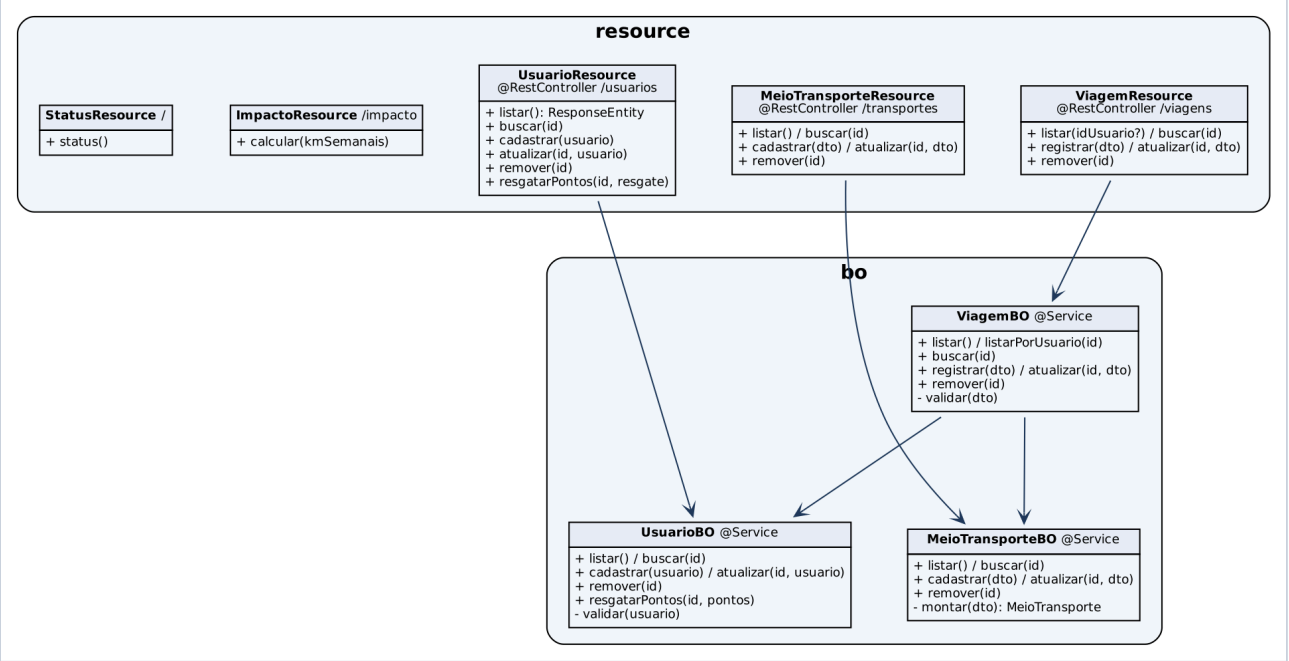


Figura 3 — Camadas resource e bo: cada endpoint delega ao BO correspondente; ViagemBO reutiliza UsuarioBO e MeioTransporteBO.

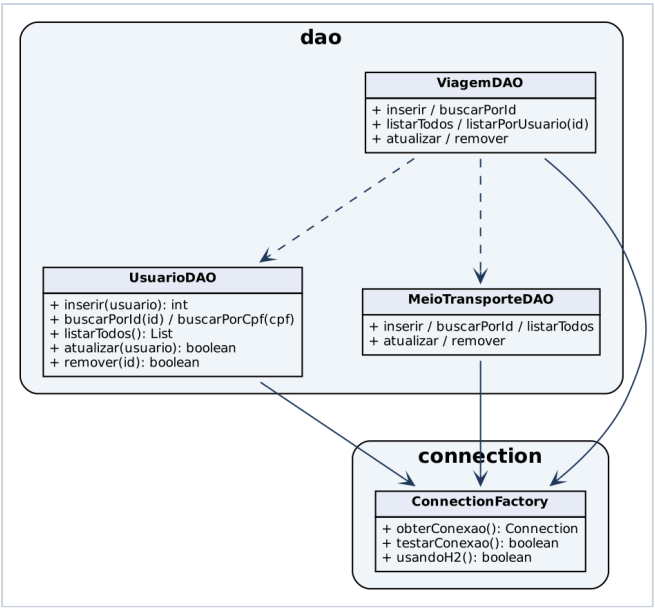


Figura 4 — Camadas dao e connection: os DAOs abrem a conexão pela ConnectionFactory; ViagemDAO usa os outros DAOs para montar o usuário e o transporte da viagem.

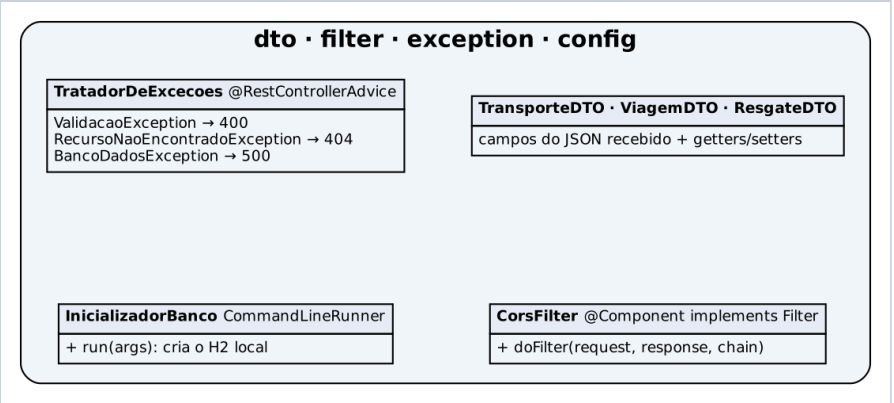


Figura 5 — Classes de apoio: DTOs, filtro de CORS, tratador de exceções e inicializador do banco local.

7. Procedimentos para rodar a aplicação

7.1 Ferramentas e versões

- **IDE:** IntelliJ IDEA (Community ou Ultimate). Também funciona no Eclipse ou VS Code com a extensão Java.
- **Java JDK:** 17 ou superior (o projeto está configurado para Java 17 no `pom.xml`).
- **Maven:** embutido no IntelliJ; para linha de comando, Maven 3.8+.
- **Banco de dados:** Oracle da FIAP (oracle.fiap.com.br) — o driver `ojdbc11` é baixado pelo Maven. Para testes sem Oracle a API sobe com um banco H2 em memória.
- **Spring Boot:** 3.3.5 (dependências `spring-boot-starter-web`, `ojdbc11`, `h2`).

7.2 Como importar o projeto no IntelliJ

1. Extraia o arquivo ZIP da entrega (ou clone o repositório do GitHub) em uma pasta de sua preferência.
2. No IntelliJ, acesse **File > Open** e selecione a pasta `SoulMove-Java-Sprint4` (a que contém o `pom.xml`).
3. Confirme em **Trust Project**. O IntelliJ reconhece o projeto Maven e baixa as dependências (aguarde a sincronização).
4. Verifique em **File > Project Structure > Project** se o SDK aponta para o JDK 17 (ou superior).

7.3 Configuração do banco de dados

1. Conecte-se ao Oracle da FIAP (SQL Developer) com seu RM e senha.
2. Execute o script `database/script_banco_soulmove.sql`: ele cria as três tabelas, as sequences e insere dois usuários e três meios de transporte de exemplo.
3. Informe as credenciais por variáveis de ambiente — elas **não** ficam no código-fonte:

   ```
   DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:ORCL
   DB_USUARIO=RM000000
   DB_SENHA=sua_senha
   ```

   No IntelliJ: **Run > Edit Configurations > Environment variables**. No terminal: `set` (Windows) ou `export` (Linux/macOS) antes de executar.
4. Se `DB_URL` não for informada, a aplicação sobe com o banco H2 em memória já populado (`src/main/resources/database/script_h2.sql`) — útil para testar sem o Oracle.

7.4 Passo a passo da execução

1. Execute a classe `br.com.soulmove.SoulMoveApplication` (botão Run do IntelliJ) ou, no terminal, `mvn spring-boot:run`.
2. No console deve aparecer `[Banco] Usando Oracle: ...` (ou a mensagem do H2) e `Tomcat started on port 8080`.
3. Abra `http://localhost:8080/` no navegador: a resposta JSON deve mostrar `"bancoDisponivel": true`.

4. Para percorrer todos os endpoints, com a API no ar, rode `bash testes/testar_api.sh http://localhost:8080` (Git Bash no Windows). O script executa o CRUD completo, os casos de erro, o resgate de pontos e a calculadora, e informa quantos testes passaram.
5. Para gerar o jar: `mvn clean package` → `target/soulmove-api.jar`, executado com `java -jar target/soulmove-api.jar`.

7.5 Front-end

1. No repositório do site (*Sprint-Front-FIAP*), crie um arquivo `.env` com `VITE_API_URL=http://localhost:8080` (ou a URL do Render).
2. Execute `npm install` e `npm run dev`; acesse a página **Painel** no menu.

8. Deploy na nuvem e repositório

A API é publicada no **Render** como *Web Service* a partir do `Dockerfile` do projeto (build em duas etapas: Maven compila e empacota; a imagem final leva apenas o JRE 17 e o jar). As credenciais do Oracle são cadastradas como variáveis de ambiente no painel do Render, e a porta é lida de `PORT`. O arquivo `render.yaml` descreve o serviço para criação via Blueprint.

Item	Endereço
Repositório GitHub	*[preencher: https://github.com/...]*
API no Render	*[preencher: https://soulmove-api.onrender.com]*
Health check	`GET /` → `{"aplicacao": "SoulMove API", "bancoDisponivel": true, ...}`

O repositório inclui um workflow do GitHub Actions (`.github/workflows/build.yml`) que, a cada *push*, compila o projeto com JDK 17, sobe a API com o banco H2 e executa o script `testes/testar_api.sh`, servindo como evidência de que a API compila e todos os endpoints respondem.

Estrutura do projeto

```
SoulMove-Java-Sprint4/
├── pom.xml
├── Dockerfile · render.yaml · README.md
├── database/script_banco_soulmove.sql        (Oracle: tabelas, sequences e carga)
├── testes/testar_api.sh                      (teste de todos os endpoints)
├── .github/workflows/build.yml               (build automático)
└── src/main/
    ├── resources/application.properties · database/script_h2.sql
    └── java/br/com/soulmove/
        ├── SoulMoveApplication.java
        ├── connection/ConnectionFactory.java
        ├── model/      Pessoa, Usuario, MeioTransporte, Onibus, Metro, Viagem
        ├── dao/        UsuarioDAO, MeioTransporteDAO, ViagemDAO
        ├── bo/         UsuarioBO, MeioTransporteBO, ViagemBO
        ├── dto/        TransporteDTO, ViagemDTO, ResgateDTO
        ├── resource/   UsuarioResource, MeioTransporteResource, ViagemResource,
ImpactoResource, StatusResource
        ├── filter/     CorsFilter
        ├── exception/  ValidacaoException, RecursoNaoEncontradoException,
BancoDadosException, TratadorDeExcecoes
        └── config/     InicializadorBanco
```